

RENTIPID · SYSTEM DOCUMENTATION

Unified Autonomous AI Customer Service & Digital Human

Complete functional, technical, data, security, operational,
testing, governance, user, and developer documentation

AS-BUILT REPOSITORY EDITION · V1 BASELINE + CURRENT-STATE AUDIT

Prepared

13 August 2026

Repository snapshot

HEAD 88565b721d0a

Historical closure

v1 closure record, 12–13 August 2026

Classification

Internal engineering & operations reference

Contents

1. Document Control and Reading Guide

- 1.1 Purpose
- 1.2 Scope
- 1.3 Evidence Basis
- 1.4 Status Vocabulary
- 1.5 Version and Baseline

2. Executive Summary and Truthful Current-State Assessment

- 2.1 Product Intent
- 2.2 What Exists
- 2.3 What Is Not Live or Not Proven in the Present Snapshot
- 2.4 Overall Assessment

3. Product Model, Actors, and Service Boundaries

- 3.1 Product Goals
- 3.2 User and Operator Roles
- 3.3 Bot Catalog
- 3.4 External Authorities
- 3.5 No-Human Routine-Support Principle

4. Architecture

- 4.1 Logical Architecture
- 4.2 Component Responsibilities
- 4.3 Request and Resolution Flow
- 4.4 Next.js 16 Boundaries
- 4.5 Azure AI Service

5. Channel and User-Interface Documentation

- 5.1 Dedicated Help Workspace
- 5.2 Contextual Assistant
- 5.3 Assistant Text Mode
- 5.4 Digital Human Mode
- 5.5 Accessibility and Responsive Behavior
- 5.6 User Operating Guide

6. Session, Provider, and Conversation Architecture

- 6.1 Provider Contract
- 6.2 Session Broker Controls
- 6.3 Conversation Continuity
- 6.4 Digital Human Provider Status

7. Support Case Platform and Lifecycle

- 7.1 Canonical Case
- 7.2 State Model
- 7.3 Severity and Risk
- 7.4 Evidence and Resolution
- 7.5 Ownership Gap
- 7.6 Follow-Up and SLA

8. Tool Gateway, Authorization, and Action Controls

- 8.1 Gateway Principle
- 8.2 Risk Classes
- 8.3 Enforcement Order
- 8.4 Implemented Demonstration Tools
- 8.5 Audit Mapping

9. Deterministic Policy Engine

9.1 Policy Result Contract

9.2 Policy Catalog

9.3 Financial Safety

9.4 Safe Hold

10. Data Model and Persistence

10.1 Additive Schema

10.2 Model Catalog

10.3 Relationship Map

10.4 Retention, Deletion, and Privacy

10.5 Migration and Rollback

11. Security, Privacy, and Abuse Resistance

11.1 Control Layers

11.2 Prompt and Output Controls

11.3 Authentication, RBAC, and Ownership

11.4 Confirmation and Step-Up

11.5 Secrets and Data Minimization

11.6 Rate Limits and Distributed Controls

11.7 Security Test Record

12. Configuration, Administration, and Monitoring

12.1 Database Settings

12.2 Environment Settings

12.3 Administration Screen

12.4 Monitoring Screen

12.5 Telemetry and Health

13. Diagnostics, Resilience, and Fallback

13.1 Diagnostic Checks

13.2 Circuit Breakers

13.3 Text-Only Degraded Mode

13.4 Failure UX

14. Testing and Evidence

14.1 Test Program

14.2 Evidence Ledger

14.3 Test Quality Caveats

14.4 Historical Acceptance

14.5 Recommended Fresh Acceptance

15. Deployment, Release, and Rollback

15.1 Intended Topology

15.2 Deployment Blocker: Route Migration

15.3 Feature-Flag Activation Order

15.4 Rollback

15.5 Operational Runbook

16. Governance, Closure, and Change Control

16.1 Architecture Locks

16.2 Accepted Scope and Limitations

16.3 Controlled Artifacts

16.4 Change Requests

17. Developer Guide

17.1 Key Paths

17.2 Local Review Workflow

17.3 Implementation Rules

17.4 Adding a Tool

17.5 Adding a Provider

17.6 Adding Knowledge

18. Operator and Administrator Guide

18.1 Daily Checks

18.2 Safe Configuration Changes

18.3 Incident Triage

18.4 Support Case Operations

19. Requirements Traceability Summary

20. Gap Register and Recommended Roadmap

20.1 Release-Blocking Gaps

20.2 Digital Human Activation Gaps

20.3 Quality and Operations Gaps

20.4 Recommended Delivery Sequence

21. API and Interface Contracts

21.1 Recommended Chat Request

21.2 Recommended Chat Response

21.3 Error Contract

22. Glossary

23. Source Register

23.1 Controlled Module Records

23.2 Implementation and Data Sources

23.3 Validation Sources

23.4 Related Governance and Diagrams

24. Final Statement

1. Document Control and Reading Guide

1.1 Purpose

This document is the authoritative consolidated reference for the RENTipid Unified Autonomous AI Customer Service & Digital Human module. It combines the module's intended architecture, controlled v1 records, actual repository implementation, operating controls, test evidence, limitations, and present integration state.

It is written for product owners, engineers, security and privacy reviewers, administrators, operations personnel, testers, and future maintainers. It is deliberately explicit where the designed system, locally tested foundations, and currently reachable production paths differ.

1.2 Scope

The module covers:

- The durable `/help` support workspace.
- The reusable contextual AI assistant embedded across public and dashboard routes.
- The Digital Human presentation mode: avatar, voice/media controls, transcript, and text fallback.
- AI configuration, role-to-bot access, prompts, guardrails, interaction logging, and monitoring.
- Secure AI sessions, conversations, support cases, evidence, tools, policy decisions, resolutions, follow-ups, knowledge-source metadata, and provider-session records.
- Deterministic support automation for bookings, cancellations, rescheduling, refunds, fees/deposits, claims, disputes, KYC, and insurance boundaries.
- Resilience, diagnostics, privacy, security, deployment, rollback, acceptance, closure, and change control.

The document does not claim that every designed capability is live. Section 2 separates historical acceptance, present tracked code, untracked working-tree implementation, simulation, and external activation dependencies.

1.3 Evidence Basis

The documentation was assembled from the repository snapshot on 13 August 2026, including:

- `docs/unified-ai-customer-service/*` controlled ledgers and acceptance records.
- `src/lib/ai/*`, including the currently untracked broker, case, context, diagnostics, policy, resilience, security, and tool subdirectories.
- `src/components/ai/*` and `src/app/help/page.tsx`.
- `src/app/api/ai/chat/route.ts` and `apps/api/src/services/aiService.ts`.
- `prisma/schema.prisma` and migration `20260812120000_add_unified_ai_foundation`.
- Phase tests `p3_test.ts` through `p11_test.ts` and `run-p12-suite.ps1`.
- Current Git branch, HEAD, historical closure commits, and working-tree status.

- Next.js 16.2.12 bundled documentation for App Router Route Handlers and Server/Client Component boundaries.

1.4 Status Vocabulary

Label	Meaning in this document
Implemented	Code or schema exists in the inspected workspace. This alone does not imply a reachable or live path.
Tracked	The artifact is present in current Git HEAD.
Untracked	The artifact exists in the working tree but is absent from current Git HEAD. It is not part of the frozen revision unless committed later.
Simulated / mock	Behavior is local, static, keyword-driven, in-memory, or otherwise not backed by a live external provider/domain service.
Contract-defined	Interfaces and boundaries exist, but the provider or integration is not activated.
Historical PASS	A controlled record reports a pass at its stated timestamp and commit; it is not a fresh validation performed for this documentation.
Current limitation	A finding from inspecting the present workspace that materially constrains live behavior.

1.5 Version and Baseline

Item	Value
Module	RENTipid Unified Autonomous AI Customer Service & Digital Human
Document edition	Repository as-built edition, 13 August 2026
Current branch	<code>feature/soc-phase4-threat-response</code>
Current HEAD	<code>88565b721d0a4e404fd6a3c6ab7d3146a394665b</code>
Current tags	<code>rentipid-ai-v1-frozen-20260813</code> , <code>rentipid-ai-v1-pre-restart-20260813</code>
Earlier closure commit named in artifacts	<code>81980e30328131dc27bce96a340458b5a7218284</code>
Starting baseline recorded by P0	<code>067ad72db92d73de58b6cf4463473c44650a173c</code>
Framework	Next.js 16.2.12, React 19.2.4, TypeScript, Prisma 6.19.3, PostgreSQL
Authentication	NextAuth 4.24.15
Primary deployment direction	Vercel frontend plus Azure backend services

2. Executive Summary and Truthful Current-State Assessment

2.1 Product Intent

RENTipid's unified support module is intended to provide one autonomous service core behind multiple presentation channels. A user should be able to ask for help from the dedicated Help workspace, an embedded contextual assistant, a PWA surface, or a Digital Human interface without creating separate logic, separate policy decisions, or parallel support databases.

The architectural principle is strong: generative AI interprets and explains, while authoritative RENTipid services and deterministic policies control sensitive outcomes. Financial decisions, KYC decisions, insurance status, booking mutation, and other consequential operations must not be invented by a model. The tool gateway is intended to enforce server-side identity, RBAC, ownership, confirmation, policy results, idempotency, auditing, and post-action verification.

2.2 What Exists

The repository contains a broad foundation:

- A reusable assistant UI, a durable Help UI, 27 role-governed bot identities, settings and monitoring screens, and 42 route files that reference the assistant component.
- An additive Prisma schema with 12 AI-domain models.
- A provider-neutral session contract, a functioning mock adapter, and a Digital Human adapter boundary.
- A session broker, case platform, tool gateway and registry, deterministic policy engine, context authorization helper, diagnostics helper, privacy guardrails, and circuit-breaker implementations in the current workspace.
- Controlled ledgers describing architecture, interfaces, tools, policy families, requirements, security review, rollback, release evidence, acceptance, closure, and scope freeze.
- Targeted local validation scripts covering P3 through P11 and a P12 aggregate runner.
- An Azure OpenAI service for embeddings and chat completion inside the Azure API application, although the inspected API routing does not expose a customer-support chat route from that service.

2.3 What Is Not Live or Not Proven in the Present Snapshot

The following are material boundaries, not cosmetic details:

Finding	Evidence	Effect
The Next.js <code>/api/ai/chat</code> route returns HTTP 410 for both GET and POST.	<code>src/app/api/ai/chat/route.ts</code> states the endpoint migrated to Azure.	Both inspected assistant UIs still POST to this route, so chat is not currently reachable through that frontend path.

Finding	Evidence	Effect
The real Digital Human adapter is not implemented.	<code>DigitalHumanProviderAdapter.initializeSession()</code> throws <code>pending credentials</code> ; <code>closeSession()</code> is empty.	No live avatar/voice provider can be created from this adapter.
The Digital Human UI simulates activation and speech.	It uses <code>setTimeout</code> , a pulsing bot graphic, and injects "Hello, I need help."	The UI is a presentation prototype, not actual WebRTC/avatar/media streaming.
The command layer always uses mock responses.	Both configured branches call <code>processMockAIRequest</code> .	Provider modes labelled OpenAI/Gemini-ready do not cause live inference in this path.
The newer shared-core implementations are untracked.	Git status shows <code>src/lib/ai/broker/</code> , <code>cases/</code> , <code>context/</code> , <code>diagnostics/</code> , <code>policy/</code> , <code>resilience/</code> , <code>security/</code> , and <code>tools/</code> as untracked.	These components are workspace artifacts, not part of current HEAD or an immutable release until committed.
Several domain tools use in-memory mock records.	<code>src/lib/ai/tools/registry.ts</code> and <code>AiContextHelper.ts</code> .	They demonstrate policies and controls but do not operate on authoritative production domain records.
Conversation continuity is modeled but not fully wired.	Schema exists, while UI requests do not send a conversation ID and the broker initializes providers with <code>conversationId: 'pending'</code> .	Cross-channel continuity is an intended contract rather than a proven end-to-end flow.
Knowledge retrieval is not implemented end to end.	<code>AiKnowledgeSource</code> metadata exists, but no active retrieval service or customer-chat RAG path was found.	FAQ/policy grounding cannot be treated as production RAG.
Historical ledgers conflict internally.	<code>MASTER_CONTROL.md</code> marks P1/P13 and final closure flags not started, while acceptance and closure files say PASS.	Closure records must be interpreted with their specific timestamp and evidence, not as a single internally consistent state machine.

2.4 Overall Assessment

The module is best characterized as a substantial locally validated foundation and UI prototype with formal v1 closure records, not as a presently operational live autonomous customer-service and Digital Human product. Its most valuable assets are the unified architecture, additive data model, deterministic policy boundary, security patterns, case lifecycle, administration surface, and test catalog. Its most urgent integration work is to restore a reachable authenticated chat endpoint, connect it to one authoritative orchestrator, replace mock domain data with server-side services, commit and review the untracked shared core, and activate a real Digital Human provider only after credentials, privacy review, and runtime acceptance.

This assessment does not alter the historical v1 records. It makes their scope and present applicability explicit.

3. Product Model, Actors, and Service Boundaries

3.1 Product Goals

The module is designed to:

- Provide natural-language, transaction-aware support.
- Minimize routine administrative involvement.
- Maintain one conversation and case history across presentation channels.
- Automate permitted resolutions while holding consequential or externally authoritative matters safely.
- Give users clear explanations, next actions, evidence requests, and status updates.
- Keep business decisions deterministic and auditable.
- Degrade from Digital Human to text rather than failing the support experience.
- Prevent a generative model from directly accessing Prisma/SQL, credentials, or unrestricted administrative operations.

3.2 User and Operator Roles

The existing RBAC catalog recognizes Guest, Renter, Individual Provider, Business Provider, Finance Admin, Admin, Compliance Admin, and Super Admin.

The assistant exposes a role-specific set of bots. Super Admin receives all configured bots. Administrative settings are accessible to Admin and Super Admin, but global/provider/maximum-permission controls are editable only by Super Admin in the inspected server action. AI logs are viewable by Super Admin, Admin, and Compliance Admin.

3.3 Bot Catalog

The current **BOTS** catalog contains 27 identities:

Family	Bots
General and onboarding	Concierge, Onboarding, Support
Trust and compliance	KYC, Category Compliance, Compliance, Security
Marketplace and booking	Listing Builder, Pricing, Booking, Agreement, Inspection
Money and disputes	Payment, Finance, Damage Claim, Dispute Review

Family	Bots
Administration and insight	Admin Copilot, Analytics
Marketing	Campaign Strategy, Listing Promotion, Caption, Hashtag, Promo Image Prompt, Video Script, Scheduler, Marketing Analytics, Influencer Outreach, WhatsApp Campaign

Prompts consistently state that the model may explain, summarize, suggest, and draft, but may not approve, make final decisions, reveal secrets, or bypass policy. The Finance Bot has an additional explicit restriction set prohibiting refund approval, payouts, bank transfers, deposit release, finance-freeze overrides, and financial execution.

3.4 External Authorities

The architecture treats the following as authoritative outside generative reasoning:

- Payment gateway for payment/refund/payout results.
- KYC provider and RENTipid identity records for identity status.
- Insurer for policy coverage, issuance, and settlement status.
- Legal or arbitration processes for matters that require them.
- RENTipid domain services and database state for bookings, listings, payments, claims, disputes, and access control.

AI may retrieve, map, summarize, or explain these results. It must not manufacture or override them.

3.5 No-Human Routine-Support Principle

The controlled architecture disallows a conventional human support queue, manual assignment, or routine takeover inside this module. Cases remain in the AI case platform. **SAFE_HOLD** is therefore not a promise that an internal human agent will take over; it means the automated process has stopped and the next action must be an approved external, administrative, legal, or system process.

This principle should be applied carefully. Legal, safety, accessibility, fraud, and high-impact exceptions still require a clearly owned escalation destination, even when that destination is outside a routine customer-service queue.

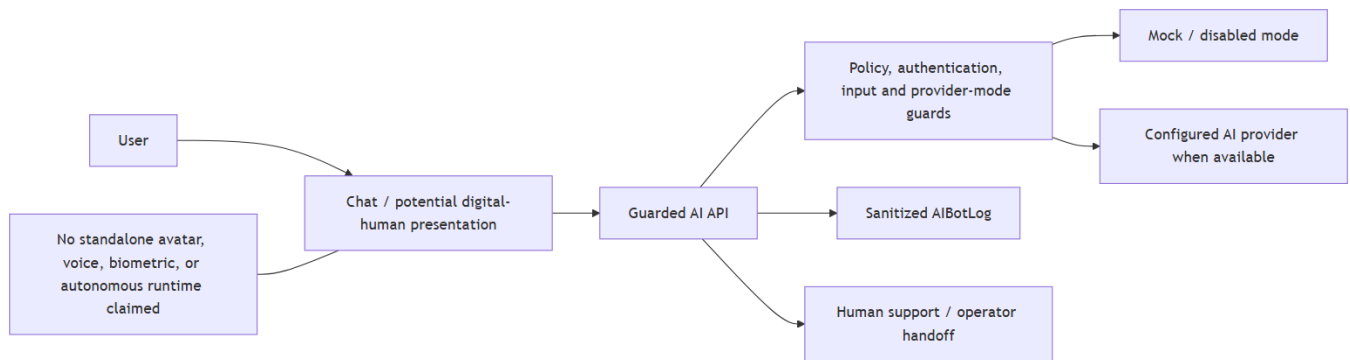
4. Architecture

4.1 Logical Architecture

The target architecture has four presentation channels and one shared core:

1. `/help` provides a durable text and case workspace.
2. The contextual assistant is embedded in route-specific screens and carries minimal authorized route/entity context.
3. Digital Human adds avatar, audio, transcript, consent, mute, and media lifecycle as presentation behavior.
4. PWA/mobile reuses the same core and changes only device/media lifecycle behavior.

All channels are intended to converge on the session broker, conversation service, support-case platform, orchestration layer, knowledge service, tool gateway, policy engine, and existing RENTipid domain services.



4.2 Component Responsibilities

Component	Responsibility	Present-state note
<code>AIAssistantButton</code>	Server-rendered feature gate; loads settings and allowed bots; renders the interactive assistant.	Tracked and embedded broadly.
<code>RentipidAIAssistant</code>	Text/Digital Human modal UI, bot selection, transcript, controls, and fallback.	Tracked; media behavior simulated.
<code>/help</code> page	Dedicated support workspace with prompts, messages, cards, and session indicator.	Tracked; submits to the 410 route.
<code>processAICommand</code>	Settings, RBAC, injection checks, guardrails, safe context, prompt selection, mock inference, output protection, and logs.	Tracked; no inspected live route calls it; inference is mock-only.
<code>AIsessionBroker</code>	Actor binding, status checks, replay protection, limits, provider initialization, expiry, termination, and fallback.	Implemented but untracked.
<code>AiCasePlatform</code>	Case creation/resume, ownership, entity links, evidence, states, resolutions, follow-up, export, closure.	Implemented but untracked.

Component	Responsibility	Present-state note
<code>AiToolGateway</code>	Tool allowlist, RBAC, replay/idempotency, confirmation, execution records, auditing, and serialization.	Implemented but untracked; several handlers are mock.
<code>AiPolicyEngine</code>	Versioned deterministic decisions with hashes and reason codes.	Implemented but untracked; thresholds include local test values.
<code>AiContextHelper</code>	Server-side authorization of route/entity context.	Implemented but untracked; mock domain data.
<code>AiDiagnosticsHelper</code>	Network, microphone, service-worker, session, provider checks and bounded repairs.	Implemented but untracked; diagnostic outcomes are simulated parameters.
<code>AiGuardrails</code>	Injection patterns, secret scrubbing, allowed-field minimization.	Implemented but untracked; simple pattern/key logic.
<code>AiCircuitBreaker</code>	Per-provider error count, text fallback, and session cost cap.	Implemented but untracked; in-memory.
<code>DigitalHumanProviderAdapter</code>	Provider SDK isolation and media session lifecycle.	Contract only; credentials and implementation pending.
<code>MockProviderAdapter</code>	Local provider-session simulation.	Tracked.

4.3 Request and Resolution Flow

The intended flow is:

```
User channel
-> authenticate and bind actor server-side
-> create/validate scoped AI session
-> authorize minimal route/entity context
-> resume or create support case when required
-> validate prompt and build safe context
-> retrieve versioned knowledge
-> generate explanation or structured tool request
-> tool allowlist + RBAC + ownership + input validation
-> deterministic policy decision
-> explicit confirmation / step-up when required
-> execute authoritative domain service
-> verify result
-> persist tool execution, resolution, message, and audit evidence
-> return text/cards/audio presentation
-> follow up or close case
```

The present UI-to-route flow stops at the migrated Vercel route, which returns 410. Sections 15 and 20 describe the required restoration path.

4.4 Next.js 16 Boundaries

The module uses the App Router. The bundled Next.js 16 guide confirms that:

- Route Handlers are `route.ts` files inside `app` and use Web `Request` / `Response` or Next extensions.
- POST handlers are not cached.
- Pages are Server Components by default.
- Interactive state, event handlers, effects, and browser APIs belong in Client Components.
- Secrets and database access belong on the server.

The current separation generally follows this model: `AIAssistantButton` and administration pages run on the server, while `RentipidAIAssistant` and `/help` are interactive Client Components. The critical defect is not the component boundary; it is the lack of a reachable replacement endpoint after the Vercel route migration.

4.5 Azure AI Service

`apps/api/src/services/aiService.ts` initializes an Azure OpenAI client using `AZURE_OPENAI_ENDPOINT` plus either `AZURE_OPENAI_API_KEY` or `DefaultAzureCredential`. It offers:

- `generateEmbeddings(text)` using `AZURE_OPENAI_EMBEDDING_DEPLOYMENT`, defaulting to `text-embedding-ada-002`.
- `generateChatCompletion(messages)` using `AZURE_OPENAI_CHAT_DEPLOYMENT`, defaulting to `gpt-4o`, temperature 0.3, maximum 1,000 tokens.

This service supports Azure AI Search indexing and potential RAG/chat, but no customer-support route invoking it was found in the inspected Azure API route set. The controlled v1 documents mention Google Gemini 2.5 Pro/Flash, while the current backend service is Azure OpenAI. Provider selection is therefore not a single settled current fact; it must be normalized in configuration and architecture records before live activation.

5. Channel and User-Interface Documentation

5.1 Dedicated Help Workspace

Route: `/help`

Purpose: a persistent, full-page AI support workspace for booking, payment, listing, and general support questions.

Visible behavior:

- Page title "RENTipid Support" and subtitle "Durable AI Workspace for your cases and questions."
- Session-status indicator.
- Suggested prompts for a booking issue and provider listing guidance.
- User and assistant message bubbles.
- Blocked-policy styling.
- Optional structured cards containing title, description, and an action button.
- Responsive text input and Enter-key submission.

Current technical behavior:

- Local state holds messages and session status.
- No durable conversation or case ID is loaded or persisted by the component.
- The component sends `botId: 'Concierge'`, while the bot catalog's exact value is `RENTipid Concierge Bot`. A restored endpoint must normalize or validate this identifier.
- The request posts to `/api/ai/chat`, which currently responds 410.
- The page imports the reusable assistant button but hides it inside `display: none` to avoid duplicate presentation.

5.2 Contextual Assistant

The reusable assistant is referenced from 42 App Router page files. These include the homepage, browse, contact, how-it-works, public listing, prohibited items, Help, renter/provider/business/admin/compliance/finance/super-admin dashboards, booking details, inspections, claims, disputes, KYC, listings, finance ledgers, and marketing screens.

The server component:

- Hides the assistant if global AI or the module is disabled.
- Determines role-allowed bots.
- Removes bots disabled by configuration.
- Passes module, record ID, role, available bots, and disclaimer to the client component.

Many call sites do not pass `userRole` or `recordId`. They therefore receive Guest bot access and generic context unless their parent supplies those values. A systematic integration review should explicitly bind authenticated role and authoritative entity IDs for every embedded route.

5.3 Assistant Text Mode

Text mode provides:

- Bot selection from the allowed set.
- A configurable disclaimer.
- Chat history held in client state.
- Clear-chat action.
- Enter and send-button input.
- Loading animation and blocked-response styling.
- A fallback message when the API request fails.

Clearing the chat only clears browser component state; it does not delete persisted conversation/case data because the current component is not connected to those stores.

5.4 Digital Human Mode

The inspected UI includes:

- A Digital Human header and a visual avatar placeholder.
- Mode switching between text and Digital Human.
- Microphone consent state.
- Mute/unmute, simulated microphone, and end-session controls.
- A live-transcript overlay.
- An explicit “Continue in Text” failure path.

However, the current mode does not request browser microphone permission, establish WebRTC, stream audio, receive synthesized speech, render a real provider avatar, or call the session broker.

`startDigitalHuman()` changes state after one second, and `simulateSpeech()` injects a fixed text prompt after two seconds. This must be presented to users only as a prototype until the provider integration is complete.

5.5 Accessibility and Responsive Behavior

The targeted P5 script checks responsive Tailwind classes, an assistant launcher `aria-label`, keyboard submission, captions, and controls. These are useful structural checks. They are not a substitute for browser acceptance with screen readers, focus trapping, Escape behavior, visible focus, color contrast, reduced-motion behavior, caption accuracy, microphone-denial flows, and touch-device testing.

5.6 User Operating Guide

For the current text interface:

1. Open Help or an assistant-enabled page.
2. Open the AI assistant.
3. Select an available bot if more than one is shown.
4. Read the disclaimer and avoid entering credentials, card security codes, or unnecessary personal data.
5. Ask one concrete question and include the relevant RENTipid reference only when the page has securely bound it.
6. Review generated explanations and drafts before acting.
7. For consequential actions, expect confirmation, policy evaluation, or safe hold.
8. If Digital Human fails, continue in text.
9. Treat provider, insurer, KYC, payment, and legal statuses as valid only when shown from their authoritative systems.

Current users should be informed that the inspected frontend chat endpoint is migrated and unavailable until the Azure integration is completed.

6. Session, Provider, and Conversation Architecture

6.1 Provider Contract

The provider-neutral interface defines:

```
initializeSession(context) -> provider session ID, expiry, metadata
sendAudio(audio)          -> optional outbound media
receiveAudio(callback)    -> optional inbound media
closeSession(sessionId)   -> terminate provider session
```

Session context includes user ID, conversation ID, channel (`help`, `digital_human`, `contextual`, or `pwa`), and locale.

Permanent provider credentials must remain server-side. The intended client contract is a short-lived, scoped broker token or provider token. The current broker token is a constructed string and is not shown to be cryptographically signed, persisted, scoped, or independently validated; production activation requires a real token design.

6.2 Session Broker Controls

The current workspace broker applies:

- Single-use nonce replay protection.
- Database user lookup and rejection of Suspended or Blacklisted users.
- A fallback-mode block for Digital Human.
- Provider health check.
- Daily limit of 50 sessions per user.
- Concurrent limit of 3 sessions per user.
- 15-minute idle timeout.
- A declared 12-hour absolute timeout, although the inspected implementation does not enforce it.
- Minimum `AIServiceSession` persistence.
- Provider initialization through mock or Digital Human adapter.
- Text fallback if provider initialization throws.
- Provider close attempt and session termination.

Nonce, usage, active-session, and last-active data are in process memory. They will reset on restart and will not coordinate across serverless instances or multiple replicas. Production requires a shared authoritative store with atomic limit and replay semantics.

6.3 Conversation Continuity

The schema provides `AiConversation` and `AiMessage`, including user association, active case, summary, last intent/channel, message role/channel/content, safe structured payload, and timestamps.

The interface contract requires all channels to carry the conversation ID. The inspected UIs do not do so, the broker uses a placeholder conversation ID for provider initialization, and no conversation service was found writing or summarizing messages. Therefore the data model is ready, but continuity is not currently end-to-end.

6.4 Digital Human Provider Status

The controlled provider register records:

- Provider: TBD.
- Required variables: `DIGITAL_HUMAN_API_URL` and server-only `DIGITAL_HUMAN_API_KEY`.
- Status: missing credentials.
- Approved fallback: text-only shared core.
- Production mode: degraded, with live provider runtime not validated.

Activation must include provider selection, DPIA/privacy review, biometric/voice implications, retention rules, regional processing, content and abuse policies, token exchange, transport security, availability SLOs, cost limits, runtime tests, failure injection, and deletion workflows.

7. Support Case Platform and Lifecycle

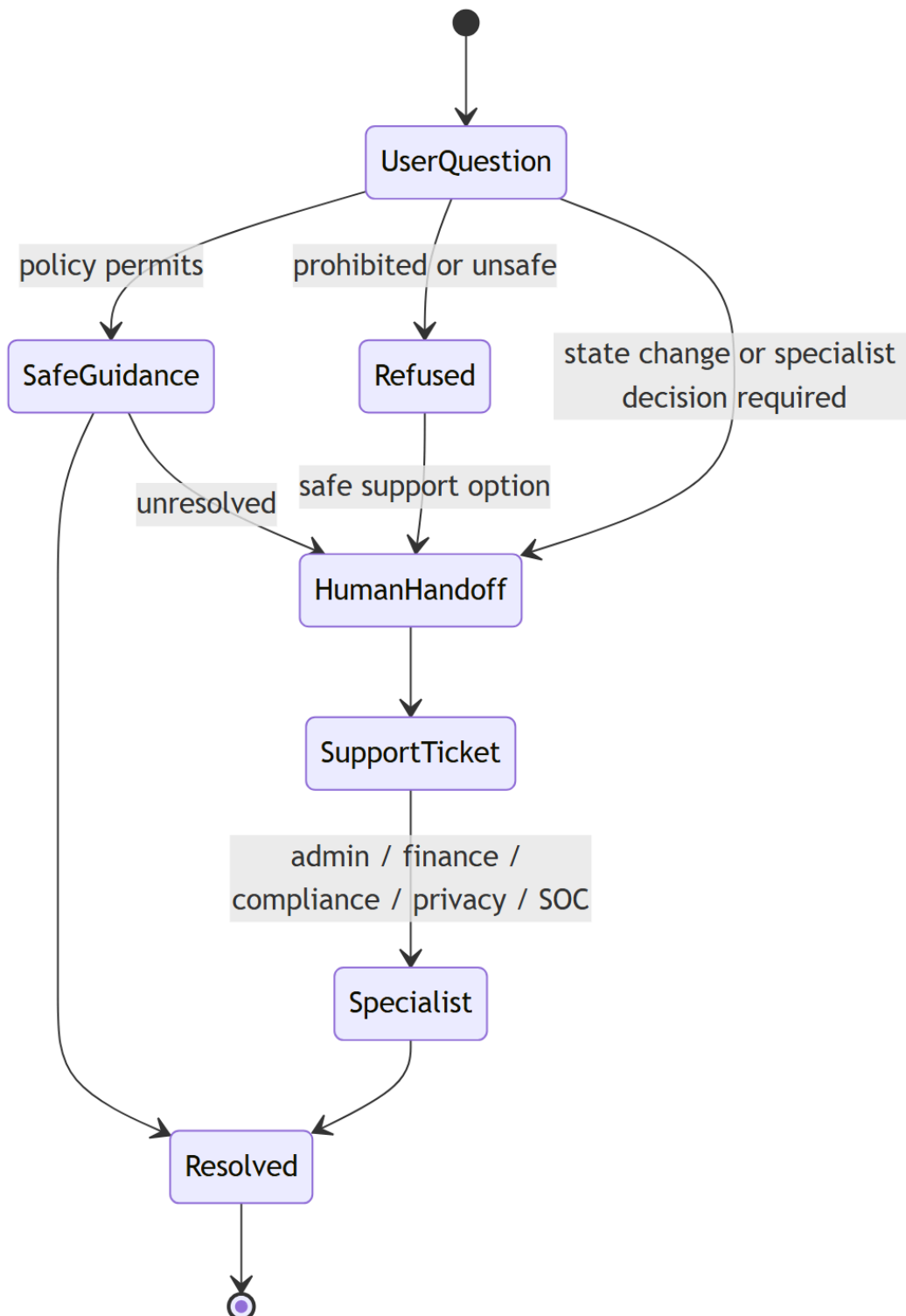
7.1 Canonical Case

AiSupportCase is the intended canonical support record. It holds a unique case number, optional user, category/subcategory, severity, risk level, state, summary, policy version, SLA deadline, activity timestamp, and resolution/closure timestamps.

The platform suppresses duplicate open cases by user/category and, where provided, matching entity link. It can create a case, link an entity, add evidence references, evaluate simple evidence completeness, update state, propose resolution, request confirmation, reconsider, schedule follow-up, finalize, close, and export the case with evidence, links, and resolutions.

7.2 State Model

State	Purpose
OPEN	Case created.
UNDERSTANDING	Intent and facts are being collected.
DIAGNOSING	Context and authoritative systems are being analyzed.
AWAITING_EVIDENCE	Required user or provider evidence is missing.
AWAITING_USER_CONFIRMATION	A binding action requires explicit confirmation.
POLICY_EVALUATION	Deterministic eligibility and limits are being evaluated.
EXECUTING	Authorized tool/domain action is executing.
VERIFYING	The system is confirming the resulting state.
SAFE_HOLD	Automation stopped because of uncertainty, conflict, limit, or external dependency.
RESOLVED	A verified outcome has been reached.
CLOSED	The case is finalized.
SYSTEM_BLOCKED	Guardrails or policy prohibited the action.



7.3 Severity and Risk

Severity values are **low**, **medium**, **high**, and **critical**. Risk values are **safe**, **consequential**, and **external**. New cases currently default to medium severity and safe risk; production classification rules were not found and must be defined before those values drive SLA or automation.

7.4 Evidence and Resolution

Evidence records include type, secure file reference, optional description, source channel, verification status, submitting user, and timestamp. The current completeness rule only checks whether any evidence is marked verified; it does not enforce category-specific evidence sets, malware scanning, metadata verification, file ownership, retention, or integrity hashes.

Resolution records support interim/final type, proposed/confirmed/executed/failed status, policy/tool references, a user-facing explanation, and verification/closure timestamps. The platform's current `finalizeResolution()` changes the case state but does not update a resolution record or prove the authoritative domain outcome. Production closure must be transactionally tied to post-action verification.

7.5 Ownership Gap

Case ownership is checked by comparing `AiSupportCase.userId`. The entity-link method contains a comment stating that real ownership of the linked Booking, Listing, or other entity should be verified. This is an explicit implementation gap. A user who owns a case must not automatically be allowed to link an arbitrary entity ID.

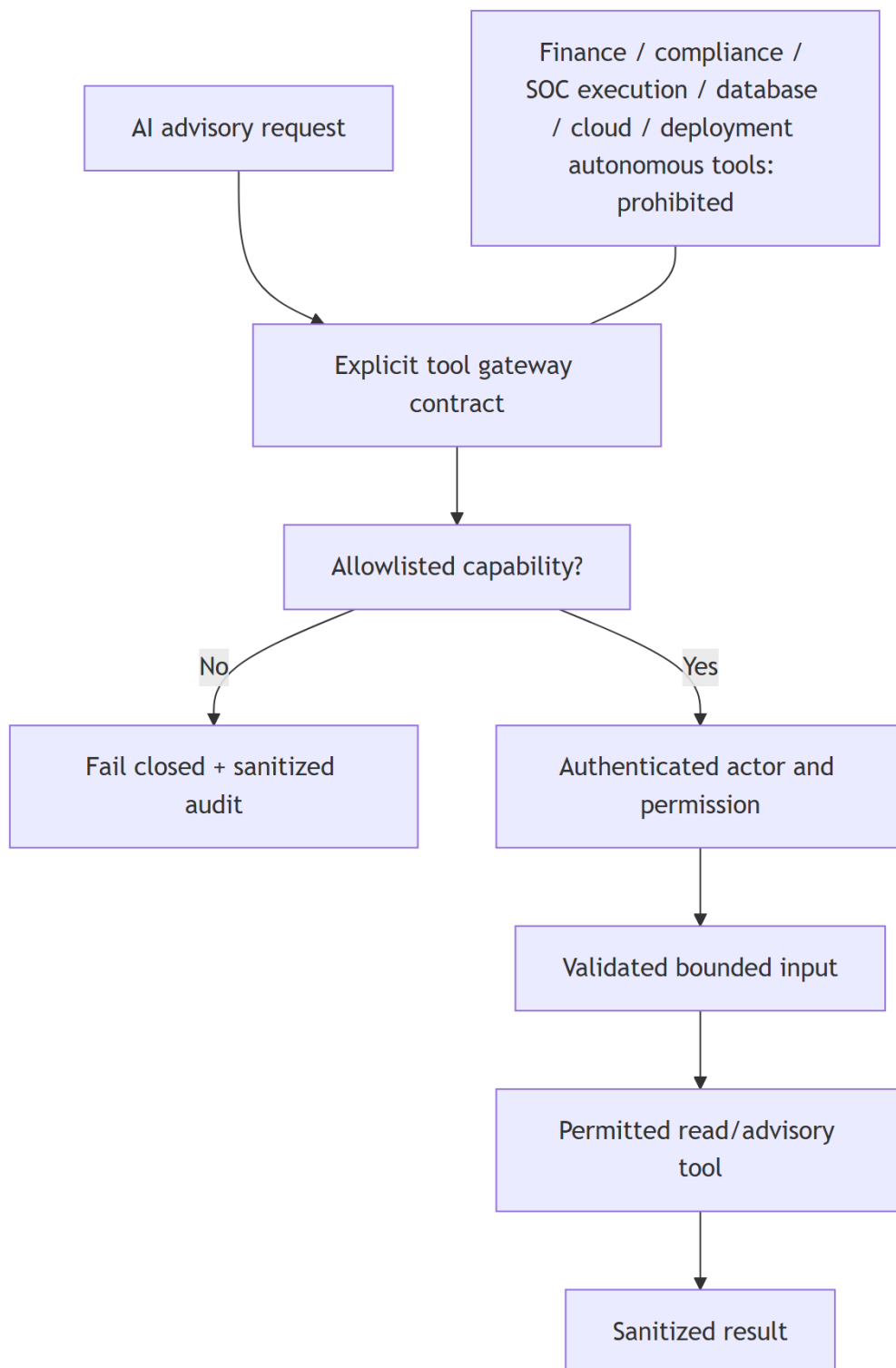
7.6 Follow-Up and SLA

`AiFollowUp` models reminders, rechecks, and re-evaluation with attempts and retry timestamps. No worker or scheduler that triggers follow-ups was found. `slaDueAt` exists but no SLA computation or breach job was found. Both are modeled capabilities awaiting operational execution.

8. Tool Gateway, Authorization, and Action Controls

8.1 Gateway Principle

The model must never receive raw database authority. It can request a named server-side tool with structured parameters and a request fingerprint. The gateway owns authorization and execution.



8.2 Risk Classes

The current TypeScript gateway defines `READ_ONLY`, `DRAFT_ONLY`, `CASE_ACTION`, `CONFIRMED_ACTION`, `POLICY_REQUIRED`, and `PROHIBITED`. Controlled documents use some different names, including `USER_CONFIRMATION_REQUIRED`, `POLICY_ENGINE_REQUIRED`, and `EXTERNAL_AUTHORITY_REQUIRED`. These vocabularies should be reconciled into one enum before production.

8.3 Enforcement Order

The inspected gateway:

1. Looks up the named tool.
2. Blocks prohibited tools and writes an audit record.
3. Resolves the user server-side.
4. Checks allowed roles.
5. Rejects replayed fingerprints for non-read operations.
6. Persists a pending execution when confirmation is missing.
7. Exposes a `requiresPolicy` marker.
8. Calls the handler.
9. Records success or denial.
10. Deep-copies the result using JSON serialization.

The policy check in the gateway itself is currently a comment/assumption. Individual handlers call the policy engine. Production should make policy invocation a non-bypassable gateway responsibility, validate tool inputs with explicit schemas, validate the AI session, verify ownership against authoritative services, bind fingerprints to canonical arguments, and persist idempotency before execution using a transactional unique constraint.

8.4 Implemented Demonstration Tools

Tool	Risk	Roles	Main control	Present implementation
<code>getBooking</code>	Read only	Renter, Provider, Admin	Ownership	In-memory mock booking.
<code>cancelBooking</code>	Confirmed action	Renter	Ownership, confirmation, cancellation policy, post-check	Mutates in-memory mock booking.
<code>adminOnlyTool</code>	Read only	Admin	RBAC	Returns demonstration secret-like data; must never be production exposed.
<code>prohibitedTool</code>	Prohibited	None in effect	Always denied	Security-control test.
<code>submitClaim</code>	Case action	Renter, Provider	Ownership and claim policy	In-memory mock claim.
<code>submitDispute</code>	Case action	Renter, Provider	Ownership and dispute policy	In-memory mock dispute.
<code>checkKyc</code>	Read only	Renter, Provider	KYC policy mapping	In-memory mock user.
<code>approveKyc</code>	Prohibited	Admin declared, but risk prohibits all	Always denied	Demonstrates that AI cannot approve identity.
<code>getInsurance</code>	Read only	Renter, Provider	Ownership and insurance status mapping	In-memory mock policy.

The controlled Tool Registry also lists `getListing`, `getPayment`, `getKycStatus`, `getInsuranceStatus`, `getCase`, and `submitCaseEvidence`, but matching registered handlers were not found in the inspected registry. Documentation consumers must not assume a ledger entry equals a currently callable tool.

8.5 Audit Mapping

The gateway's `logSecurityEvent()` currently writes to `AuditLog` as a fallback, even where test output labels it a `SecurityEvent`. Production must write the correct security-event type into the SOC ingestion path, preserve correlation IDs, avoid storing sensitive raw prompts or results, and ensure audit failure policy is explicit.

9. Deterministic Policy Engine

9.1 Policy Result Contract

Every evaluation returns a decision (`approved` , `denied` , or `hold`), eligibility, stable reason code, policy version, optional calculated amount, confirmation requirement, step-up requirement, safe-hold flag, and optional next action.

Inputs are SHA-256 hashed and the decision is persisted in `AiPolicyDecision` . Because `JSON.stringify` ordering is used directly, production should canonicalize inputs before hashing to guarantee stable cross-runtime identity.

9.2 Policy Catalog

Family	Core rule	Confirmation	Step-up / hold	Current note
Cancellation	Confirmed booking more than 24 hours from start is approved; disputed/unknown holds; otherwise denied.	Approved cancellation requires confirmation.	No step-up in current rule.	Approved result returns <code>calculatedAmount: 100</code> , described as percentage-like but not strongly typed.
Rescheduling	Available date approved; unavailable denied.	Approval requires confirmation.	None.	Current code applies a numeric change fee of 10.
Refund	Provider fault approved for requested amount; renter fault denied; unknown fault holds.	Approval requires confirmation.	Amount over 500 requires step-up.	Currency is not represented in the result.
Fees/deposits	Risk score over 80 produces 20% deposit; otherwise 10%.	No.	High risk requires step-up.	Risk-score provenance is not shown.
Claim	Conflict/incomplete evidence holds; amount over 1,000 holds; otherwise approved.	Approval requires confirmation.	Over 1,000 requires step-up and external process.	1,000 is explicitly a local test value.
Dispute	Conflict/incomplete evidence or amount over 500 holds; otherwise approved.	Approval requires confirmation.	Current over-limit helper does not set step-up.	500 is explicitly a local test value.
KYC	Verified approved; rejected denied; other status holds.	No.	Unknown status holds.	AI cannot approve KYC.
Insurance	Active approved; inactive/unknown holds.	No.	Non-active status holds.	Insurer remains authoritative.

9.3 Financial Safety

The closure records explicitly say that 1,000/500 claim and dispute thresholds are test values configurable in the database. The current code hard-codes them and uses dollar-style comments despite RENTipid's Philippines context. Production policy must specify currency, monetary units, rounding, effective version, jurisdiction, tax/fee treatment, authority, approval tiers, and database-controlled thresholds.

Generative output must never be treated as a binding calculation. A policy decision should be tied to the exact transaction snapshot, and execution should fail closed if the snapshot changes before mutation.

9.4 Safe Hold

Safe hold is returned for unknown/conflicting state, missing/conflicting evidence, exceeded automation thresholds, unknown KYC, or inactive/unknown insurance. A safe hold prevents the handler from proceeding. The operational next destination must be defined per reason code; "escalate" without an owner, deadline, and user-visible status is incomplete.

10. Data Model and Persistence

10.1 Additive Schema

Migration `20260812120000_add_unified_ai_foundation` creates 12 tables and adds only three foreign keys: session-to-user, conversation-to-user, and message-to-conversation. It also adds uniqueness/index controls for case number, tool idempotency, knowledge slug, provider-session reference, cases, entity links, and policy decision lookup.

10.2 Model Catalog

Model	Purpose	Key integrity controls	Notable limitations
<code>AiServiceSession</code>	Channel session and provider reference.	Optional user FK.	<code>conversationId</code> is not a FK; no expiry/last-active columns.
<code>AiConversation</code>	Cross-channel conversation metadata.	Optional user FK.	<code>activeCaseId</code> is not a FK.
<code>AiMessage</code>	User, assistant, system, and tool turns.	Conversation FK with cascade delete.	<code>sessionId</code> is not a FK; raw <code>content</code> retention policy undefined.
<code>AiSupportCase</code>	Canonical case record.	Unique case number; user/status/activity index; optional user FK.	Related case tables have no case FKs.
<code>AiCaseEntityLink</code>	Links case to Booking, Listing, Payment, etc.	Case and entity indexes.	No case FK; polymorphic entity integrity is application-only.
<code>AiCaseEvidence</code>	Evidence reference and verification status.	None beyond primary key.	No case/user FKs; no content hash or storage-integrity metadata.
<code>AiToolExecution</code>	Authorization, policy, confirmation, execution, and verification trace.	Unique optional idempotency key.	No session/case FK; one nullable unique key requires careful provider behavior.
<code>AiPolicyDecision</code>	Versioned deterministic decision with input hash.	Composite lookup index.	No case FK; no explicit actor, currency, or transaction-snapshot version.
<code>AiResolution</code>	Proposed/final outcome.	None beyond primary key.	No FKs to case/policy/tool; no created timestamp.
<code>AiFollowUp</code>	Scheduled reminder/recheck/re-evaluation.	None beyond primary key.	No case FK; no worker lease/dead-letter fields.
<code>AiKnowledgeSource</code>	Versioned knowledge-source metadata.	Unique slug.	No content/chunk/index model and no retrieval implementation.
<code>AiProviderSession</code>	External provider session reference and expiry.	Unique provider reference.	No FKs to user/session; provider-specific consent and region metadata absent.

10.3 Relationship Map

The intended relationship is User → Session/Conversation/Case, Conversation → Messages, Case → Entity Links/Evidence/Tool Executions/Policy Decisions/Resolutions/Follow-Ups, and Session → Provider Sessions. Only part of that graph is enforced by the migration. Application code must not be relied on as the sole integrity mechanism for durable evidence and financial/support history without a documented reason.

10.4 Retention, Deletion, and Privacy

The schema does not encode retention policy. Operations must define:

- Conversation and raw prompt retention.
- Case/evidence retention by category and legal requirement.
- Provider audio/video retention and whether media is stored at all.
- Policy and tool audit retention.
- User deletion/anonymization while preserving required financial/audit records.
- Subject-access/export behavior.
- Deletion propagation to external providers and vector/search indexes.

10.5 Migration and Rollback

The controlled rollback plan uses an additive migration strategy:

1. Deploy code with features off.
2. Run `prisma migrate deploy`.
3. Verify database health.
4. Enable feature flags gradually.
5. Roll back application deployment to a prior stable SHA if required.
6. Use a forward-fix migration rather than destructive down migration in production.

Stated triggers include greater than 1% 5xx rate, read latency over 500 ms, and automated security alarms. Managed backup, PITR, and read-replica synchronization are listed as prerequisites/assumptions and must be verified in the target environment rather than inferred from documentation.

11. Security, Privacy, and Abuse Resistance

11.1 Control Layers

The design uses defense in depth:

- Server-side authentication and actor binding.
- Role-to-bot access controls.
- Global, module, and bot feature switches.
- Prompt injection detection and blocked-keyword guardrails.
- Server-side tool registry and RBAC.
- Entity ownership checks.
- Confirmation and deterministic policy boundaries.
- Replay/idempotency controls.
- Secret scrubbing and allowed-field serialization.
- Output protection.
- Audit and security-event integration.
- Provider circuit breaker, limits, and text fallback.

11.2 Prompt and Output Controls

There are two guardrail implementations:

- `checkGuardrails()` blocks a list of sensitive action phrases such as approving KYC, publishing listings, verifying payment, releasing deposits, deciding disputes, refunds, payouts, keys, and bypass attempts.
- `AIGuardrails.detectInjection()` detects a short set of common patterns and offers recursive secret-key scrubbing plus allowlisted fields.
- The command layer also invokes `AIGuard` from the SOC detection subsystem for input and output protection.

Pattern matching is a useful layer but is not a complete prompt-injection defense. Production must treat all retrieved content, tool output, attachments, and provider metadata as untrusted; strictly separate instructions from data; allow only typed tools; cap arguments; and make authorization independent of model wording.

11.3 Authentication, RBAC, and Ownership

The strongest rule is that user ID and role must come from a validated server session, never the request body. The legacy `AIRequest` interface accepts `userRole` and `userId`, so any future route invoking it must overwrite those fields from server authentication.

The tool gateway resolves the user from the database and checks role, but it does not validate the supplied `sessionId`. Context and tool demonstrations use mock databases. Production must centralize ownership in reusable domain authorization services to prevent IDOR across Booking, Listing, Payment, Claim, Dispute,

Case, Evidence, and Insurance records.

11.4 Confirmation and Step-Up

Confirmation must be explicit, scoped to a canonical action preview, short-lived, non-replayable, and invalidated if material inputs change. A Boolean `userConfirmed` argument alone is insufficient for a live consequential action.

Step-up authentication is represented in policy results but no authentication challenge or grant-consumption mechanism was found in this module. High-value actions must consume a server-issued step-up grant tied to actor, action, resource, amount, currency, and expiry.

11.5 Secrets and Data Minimization

Permanent Digital Human and AI provider keys must never enter a Client Component or client JSON. The server-only boundary should issue narrowly scoped short-lived tokens. Logging currently stores prompts and a response summary in `AIBotLog`; this creates privacy and prompt-injection-evidence retention obligations. Redaction must happen before persistence, not only before provider transmission.

The current recursive scrubber checks key names containing password, token, secret, credit card, SSN, or CVV. It will not reliably detect sensitive values embedded in free text. Production requires data classification and structured redaction for Philippine identifiers, phone/email/address data, payment references, KYC documents, and claim evidence.

11.6 Rate Limits and Distributed Controls

The Azure API has general and strict rate-limit middleware, but the inspected AI frontend route is a 410 handler and the shared-core broker's limits are in memory. A live AI route needs authenticated per-user, per-IP/risk, per-session, and provider-budget controls in a shared store, with stricter limits for session creation, tools, uploads, and confirmation attempts.

11.7 Security Test Record

The controlled P11 record reports passes for prompt injection, hidden instructions, prohibited tools, cross-user access/mutation, role escalation, actor spoofing, ownership bypass, privacy minimization, secret exposure, replay, duplicate mutation, confirmation bypass, step-up boundary, provider outage, circuit breaker, text fallback, and usage limits.

These are targeted local/structural tests. Several exercise in-memory mocks or call methods directly. They do not prove network authentication, browser isolation, distributed replay prevention, live provider handling, data exfiltration resistance, or production infrastructure configuration.

12. Configuration, Administration, and Monitoring

12.1 Database Settings

`getAISettings()` loads `SystemSetting` records whose keys begin with `ai_`. Principal settings include:

Setting	Purpose	Default behavior
<code>ai_global_enabled</code>	Master assistant switch.	Enabled.
<code>ai_logging_enabled</code>	Interaction-log switch.	Enabled.
<code>ai_mock_mode_enabled</code>	Forces mock behavior.	Enabled.
<code>ai_provider_mode</code>	<code>mock</code> , <code>openai</code> , <code>gemini</code> , or <code>disabled</code> .	<code>mock</code> .
<code>ai_max_permission</code>	Nominal maximum permission level.	3, draft-only.
<code>ai_response_style</code>	Response style.	<code>Simple</code> .
<code>ai_disclaimer_text</code>	User-visible disclaimer.	Assist/summarize, no final decisions.
<code>ai_module_*_enabled</code>	Per-module switch.	Enabled unless explicitly false.
<code>ai_bot_*_enabled</code>	Per-bot switch.	Enabled unless explicitly false.

Default-on behavior is convenient but risky for newly introduced modules/bots. Production configuration should prefer explicit registration and deny-by-default for high-impact contexts.

12.2 Environment Settings

The tracked AI environment contract includes:

Variable	Meaning
<code>DIGITAL_HUMAN_API_URL</code>	Server-side Digital Human provider URL.
<code>DIGITAL_HUMAN_API_KEY</code>	Server-only provider credential.
<code>AI_FALLBACK_MODE_ENABLED</code>	Prevent Digital Human session creation and use text fallback.
<code>AI_PROVIDER MOCK_ENABLED</code>	Select mock provider adapter.
<code>AI_MAX_SESSION_DURATION_MS</code>	Nominal maximum session duration; not consumed by the inspected broker.

The Azure backend additionally uses `AZURE_OPENAI_ENDPOINT`, `AZURE_OPENAI_API_KEY` or managed identity, `AZURE_OPENAI_EMBEDDING_DEPLOYMENT`, and `AZURE_OPENAI_CHAT_DEPLOYMENT`.

12.3 Administration Screen

Admin/Super Admin settings provide:

- Global activation.
- Provider mode.
- Maximum permission level.
- Disclaimer text.
- Module activation switches for public, registration, listing, booking, payment, agreement, inspection, dispute, finance, admin, and compliance.
- Availability switches for every bot.

Only Super Admin may alter the global switch, provider mode, permission maximum, and disclaimer. Admin can alter module and bot switches. Changes are upserted and sent through administration audit/security ingestion.

The UI warning says sensitive actions require authorized human approval, while the module architecture says no routine human service. This is not necessarily contradictory: approval is an authorization control, not a support queue. Operational documentation should name which roles perform which approvals.

12.4 Monitoring Screen

The AI logs dashboard displays total interactions, blocked interactions, filters, and the 50 most recent **AIBotLog** records with user, role, bot, module, prompt, and status. The visible “Export Logs” button has no inspected action. Production should add pagination, time/risk filters, redacted export, access auditing, retention controls, and correlation to tool, policy, case, provider, and security events.

12.5 Telemetry and Health

The tracked telemetry service holds per-session token usage in memory, defaults to 10,000 tokens, and writes token/cost information to console. The untracked circuit breaker uses arbitrary cost units with a maximum of 50. These overlapping mechanisms must be consolidated.

AiHealthService tracks provider health in memory and opens after five failures for 60 seconds. The untracked breaker opens after three errors. A single configurable distributed resilience policy is needed, with metrics for sessions, provider latency/errors, fallback rate, token/cost, tool authorization, holds, confirmation abandonment, case resolution, and user satisfaction.

13. Diagnostics, Resilience, and Fallback

13.1 Diagnostic Checks

The helper models network, microphone permission, service worker, session, and provider health. It recommends `USE_TEXT_FALLBACK`, `REGISTER_SW`, or `RECREATE_SESSION`. The self-repair method maps those recommendations to bounded status strings.

This is a safe pattern: diagnose first, attempt only predefined reversible repairs, and never allow arbitrary model-generated repair code. Present checks receive Boolean arguments rather than probing actual browser/server state.

13.2 Circuit Breakers

Provider errors increment a counter; successful calls reset it; an open circuit uses a fallback task. Production should add half-open probes, time windows, concurrency protection, distributed state or instance-aware design, provider-specific thresholds, retry budgets, and telemetry.

13.3 Text-Only Degraded Mode

Text fallback is a central requirement. It must preserve authentication, conversation ID, active case, policy state, pending confirmation, and user-visible explanation. The current UI switches visual mode and the broker can report `fallbackToText`, but end-to-end continuity is not yet wired.

13.4 Failure UX

Required behavior by failure:

Failure	User experience	System action
Microphone denied	Explain and remain in text.	Do not repeatedly request permission.
Avatar provider unavailable	Offer immediate text continuation.	Open circuit; preserve conversation/case.
AI provider unavailable	Provide non-generative status and safe navigation.	Do not execute tools based on incomplete reasoning.
Session expired	Ask user to reauthenticate/recreate session.	Invalidate pending confirmations/tokens.
Domain authority unavailable	Show safe hold with retry/status information.	Persist case and follow-up.
Mutation timeout	Do not blindly repeat.	Query authoritative status using idempotency reference.
Usage limit reached	Explain limit and available non-AI support paths.	Stop provider calls; preserve case.

14. Testing and Evidence

14.1 Test Program

The module's targeted scripts cover:

Phase	Scope
P3	Schema relations, duplicate domains, environment validation, mock provider, health/circuit foundation.
P4	Session creation, unauthorized/suspended denial, replay, concurrent/daily limits, actor binding, termination, feature flag, provider fallback, secret exposure, expiry/cleanup.
P5	Help UI, placeholder removal, text interaction, Digital Human UI, mic consent, transcript, controls, context marker, fallback, responsive layout, accessibility smoke, client secret scan.
P6	Case creation/ownership/lifecycle, evidence, follow-up, resolutions, reconsideration, duplicate suppression, cross-channel resume, no-human-queue structure.
P7	Tool actor resolution, input claim, unauthorized/RBAC/ownership denial, prohibited tools, audit mapping, confirmation, policy, verification, privacy serialization, idempotency/replay.
P8	Deterministic cancellation/rescheduling/refund/deposit logic, version/reason codes, step-up, thresholds, safe hold, hashing, confirmation, gateway integration, generative override.
P9	Claims/disputes ownership, evidence holds, thresholds, reconsideration, KYC mapping/prohibition, insurance authority, audit/security, duplicate-channel structure.
P10	Context authorization, microphone/text fallback, service worker/PWA, provider/session diagnostics, bounded repairs, continuity claims, mutation retry safety.
P11	Prompt/output security, prohibited tools, cross-user access, escalation/spoofing, privacy/secret controls, replay/confirmation/step-up, outage fallback, limits, no-human-service.
P12	Runs P3–P11 and compares pre/post SHA-256 digest of <code>src/lib/ai</code> to detect modifications during validation.

14.2 Evidence Ledger

The controlled evidence ledger records P3 through P12 passes between 12 and 13 August 2026. It also records corrections:

- Digital Human provider runtime proof pending credentials.
- Text fallback architecture validated; early runtime proof not yet proven, later targeted tests report local fallback pass.
- P5 UI structure validated, not browser runtime.
- P10 Capacitor runtime not proven.
- Claim/dispute thresholds are local test values.

14.3 Test Quality Caveats

Several scripts are executable demonstrations rather than framework assertions. P5 performs source-string checks. Some tests print PASS for structural or simulated conditions. P7 calls privacy serialization a PASS based on a JSON copy, and labels AuditLog fallback as SecurityEvent. P10 reports some behaviors as architecturally covered or simulated. These limitations should be preserved in any acceptance interpretation.

14.4 Historical Acceptance

`FINAL_ACCEPTANCE.md` records local E2E, migration, required data, acceptance, production build, deployment configuration, rollback, and deployment readiness as PASS at HEAD `81980e3...`, with approved degraded Digital Human mode and deferred Capacitor. `CLOSURE_CERTIFICATE.md` records completion, validation, acceptance, local functionality, deployment readiness, manifest, security/privacy, and rollback as PASS.

`MASTER_CONTROL.md`, however, still says P1 and P13 were not started and global completion/closure/freeze statuses were not started. This inconsistency should be corrected through controlled change management rather than silently rewritten.

14.5 Recommended Fresh Acceptance

Before live activation, execute:

1. Clean-checkout build and type/lint verification.
2. Fresh disposable database migration and seed.
3. Framework-based unit and integration tests, not source-string checks.
4. Authenticated HTTP tests for the new Azure chat/session/tool endpoints.
5. Browser tests for Help, contextual assistant, Digital Human consent/media, accessibility, responsive behavior, and fallback.
6. Distributed idempotency, replay, quota, and circuit-breaker tests.
7. Real domain-service tests for booking, payment, claims, disputes, KYC, and insurance boundaries.
8. Live-provider sandbox tests with data-minimization and deletion proof.
9. SOC event/correlation validation.
10. Load, latency, cost, outage, rollback, and recovery exercises.

15. Deployment, Release, and Rollback

15.1 Intended Topology

The present repository direction is:

- Next.js frontend and interactive components on Vercel.
- Azure API backend in `apps/api` / Azure Container Apps.
- PostgreSQL through Prisma.
- Azure OpenAI and Azure AI Search services.
- Application Insights and Azure secret/managed-identity support.
- A future Digital Human provider behind the provider adapter.

Earlier module documents state Google GenAI and Vercel serverless as the AI path. These are historical descriptions and must be reconciled with the Azure migration before release.

15.2 Deployment Blocker: Route Migration

The Next.js route deliberately returns 410 and tells callers to use `src/lib/api-client.ts` / `azureFetch`. The two customer-facing UIs have not been updated accordingly. A correct repair should:

- Define an authenticated Azure customer-support endpoint.
- Route frontend requests through the approved API client.
- Derive actor/role server-side.
- Validate request/body size and schema.
- Create/validate session and conversation.
- Invoke the one orchestrator.
- Preserve correlation and idempotency IDs.
- Return a versioned response contract.
- Map failures to safe, non-sensitive user messages.

15.3 Feature-Flag Activation Order

Recommended sequence:

1. Deploy schema with all new features disabled.
2. Validate database and application health.
3. Enable internal text mock mode for authorized testers.
4. Enable authenticated text provider mode for a small cohort.
5. Enable read-only tools.
6. Enable case/evidence workflows.
7. Enable confirmed deterministic mutations one family at a time.

8. Validate fallback and circuit behavior.
9. Enable Digital Human for internal users, then limited cohort.
10. Expand only while error, safety, privacy, cost, and resolution metrics remain within approved limits.

15.4 Rollback

Application rollback should revert the Vercel/Azure revision and disable feature flags. Additive schema remains in place for compatibility. Production database down migrations are prohibited; defects use forward fixes. Before migration, verify backup and PITR. After deployment, smoke-test health, login, booking, AI session initialization, Help, tool authorization, and fallback.

15.5 Operational Runbook

When error/fallback rates rise:

1. Identify frontend, Azure API, database, AI provider, Digital Human provider, or domain authority as the failing boundary.
2. Turn on text fallback or disable affected tools/provider mode.
3. Preserve correlation IDs, execution records, policy decisions, and case state.
4. Reconcile timed-out mutations with authoritative systems before retry.
5. Notify users through case status/follow-up without exposing internal errors.
6. Roll back application revision if release-correlated.
7. Use forward-fix migration for schema defects.
8. Record the incident, root cause, recovery evidence, and change request.

16. Governance, Closure, and Change Control

16.1 Architecture Locks

The v1 architecture locks:

- One shared orchestrator.
- One canonical AI case platform.
- One typed AI tool gateway.
- One versioned knowledge service.
- Existing deterministic RENTipid services as authoritative.
- Presentation-only channel variation.
- Additive use of the existing PostgreSQL/Prisma database.
- No routine human support queue.
- Provider SDK isolation and server-side credentials.

16.2 Accepted Scope and Limitations

Historical frozen scope includes Help, AI APIs/webhooks, session/case/tool/policy services, Google GenAI labels, AI Prisma models, security controls, P1–P12 tests, Digital Human adapter contract, degraded provider mode, contextual AI, and domain orchestration. Approved limitations are:

- Live Digital Human provider runtime not validated.
- 1,000/500 claim/dispute thresholds are test values.
- Capacitor UI/runtime deferred from v1.

The present audit adds the current 410 endpoint, mock-only command path, untracked shared core, and mock domain tools as material current-state limitations.

16.3 Controlled Artifacts

The module directory contains baseline, architecture, ownership, implementation, interface, requirements, tool, policy, case-state, provider, security/privacy, production activation, rollback, evidence, acceptance, closure, scope, digests, and artifact manifest records.

This new systematic document and PDF are documentation additions. They do not by themselves change the frozen software baseline or re-certify the module. If incorporated into the controlled manifest, hashes and closure/change records must be updated through an approved change request.

16.4 Change Requests

The frozen-scope record requires future changes to use **UAICS-DH-CR-###**. A change request should include objective, affected artifacts, threat/privacy assessment, migration impact, compatibility, test plan, rollback, evidence, approvers, resulting commit/tag, and manifest update.

17. Developer Guide

17.1 Key Paths

Area	Paths
UI	<code>src/components/ai/*</code> , <code>src/app/help/page.tsx</code>
Legacy orchestrator	<code>src/lib/ai/ai-command-layer.ts</code> and sibling <code>ai-*</code> files
Shared-core workspace	<code>src/lib/ai/broker</code> , <code>cases</code> , <code>context</code> , <code>diagnostics</code> , <code>policy</code> , <code>resilience</code> , <code>security</code> , <code>tools</code>
Provider contracts	<code>src/lib/ai/gateway/ai-contracts.ts</code> , <code>src/lib/ai/adapters/*</code>
Frontend route	<code>src/app/api/ai/chat/route.ts</code>
Azure AI service	<code>apps/api/src/services/aiService.ts</code>
Database	<code>prisma/schema.prisma</code> , migration <code>20260812120000_add_unified_ai_foundation</code>
Admin	<code>src/app/dashboard/admin/ai-settings</code> , <code>ai-logs</code> , and super-admin wrappers
Tests	<code>p3_test.ts</code> through <code>p11_test.ts</code> , <code>run-p12-suite.ps1</code>
Controlled docs	<code>docs/unified-ai-customer-service/*</code>

17.2 Local Review Workflow

Before changing Next.js code, read the relevant bundled guide in `node_modules/next/dist/docs/`. For this module, start with App Router Route Handlers, Server/Client Components, and version-16 upgrade notes.

Then:

1. Check Git status and preserve unrelated user changes.
2. Determine whether work targets the frozen v1 baseline or a new change request.
3. Reconcile tracked and untracked module files.
4. Use an isolated test database and run the repository database guard before migrations/tests.
5. Never expose environment/provider credentials to Client Components.
6. Add typed request/response schemas and server-side actor resolution.
7. Extend one authoritative service rather than creating a parallel support path.
8. Add negative authorization, replay, confirmation, policy, outage, and privacy tests.
9. Update ledgers, evidence, hashes, and operational instructions together.

17.3 Implementation Rules

- Use one shared Prisma client pattern rather than constructing clients throughout server modules.
- Replace `any` contracts with validated schemas.

- Use canonical structured IDs and currencies for monetary policies.
- Make tool handlers thin adapters over authoritative domain services.
- Persist session/quota/replay state in a shared store.
- Use transactional outbox or equivalent for audit/security/follow-up reliability.
- Do not log unredacted prompts or provider payloads.
- Do not treat model text as authorization, confirmation, policy, or post-action proof.
- Keep Digital Human media logic outside business logic.
- Ensure text fallback preserves the same conversation and case.

17.4 Adding a Tool

For every new tool:

1. Define stable name, purpose, risk class, allowed roles, input/output schema, ownership rule, policy family, confirmation/step-up need, idempotency semantics, authoritative service, audit/security events, verification, and failure mapping.
2. Register server-side only.
3. Validate the authenticated AI session and actor.
4. Fetch and authorize the target entity.
5. Evaluate deterministic policy against a versioned snapshot.
6. Obtain a scoped confirmation grant when needed.
7. Reserve idempotency transactionally.
8. Execute the domain service.
9. Read back authoritative state.
10. Persist execution and resolution evidence.
11. Return only minimized user-safe fields.
12. Test happy, denied, cross-user, replay, timeout, stale-confirmation, policy-hold, provider-failure, and audit-failure paths.

17.5 Adding a Provider

A provider implementation must support server-side configuration validation, short-lived scoped session creation, close/revocation, timeout, health, bounded retry, cost telemetry, privacy minimization, region/retention controls, and text fallback. Media permission belongs in the browser; permanent keys never do.

17.6 Adding Knowledge

Knowledge sources should be versioned, effective-dated, role-scoped, approved, and attributable. Retrieval must filter by status/effective dates/role, return citations, resist prompt injection in source content, and avoid indexing secrets or unauthorized records. Superseded knowledge must not silently remain active in search indexes.

18. Operator and Administrator Guide

18.1 Daily Checks

- Global/module/bot configuration matches the approved release.
- Text and Digital Human provider health.
- 5xx, latency, fallback, circuit-open, and session-creation trends.
- Blocked tool, RBAC, ownership, injection, replay, and confirmation events.
- Safe-hold counts by reason and unresolved age.
- Case SLA/follow-up backlog.
- Token, media-minute, and cost budgets.
- Domain reconciliation for any executed financial or booking action.

18.2 Safe Configuration Changes

Use Super Admin for global/provider/permission/disclaimer changes and Admin/Super Admin for module/bot switches. Confirm the audit/security event after saving. Enable risky capabilities gradually. Do not set a provider mode label unless the backend path, credentials, health, and runtime acceptance for that provider are complete.

18.3 Incident Triage

Use correlation among `AIBotLog`, `AuditLog`, `SecurityEvent/SOC`, `AIServiceSession`, `AiToolExecution`, `AiPolicyDecision`, `AiSupportCase`, and provider telemetry. Do not copy raw prompts or evidence into unsecured tickets. For suspected data leakage, disable affected provider/tool, preserve protected evidence, invoke the privacy incident process, and revoke provider tokens.

18.4 Support Case Operations

Even without a routine human queue, operators own system-level exceptions: provider outage, failed jobs, stuck follow-ups, reconciliation mismatches, security events, and legal/external authority handoffs. They should not override deterministic policy by editing outcomes directly. Repairs must be recorded, authorized, and verified.

19. Requirements Traceability Summary

The controlled requirements ledger lists 52 requirements across architecture, anti-duplication, authentication, RBAC, ownership, sessions, conversations, Help, Digital Human, orchestration, cases, evidence, follow-up, knowledge, tools, policy, bookings/listings/provider/payments/refunds/deposits/escrow/payouts/KYC/claims/disputes/insurance, context, PWA/Capacitor, diagnostics/self-repair, audit/security, privacy/injection/outage/limits, migration/data/E2E/acceptance/build/deployment/rollback, validation/closure/freeze, and no-human-service design.

The following current-state matrix is more precise than a single Implemented flag:

Requirement family	Design/schema	Local implementation	Reachable current UI path	Live external proof
Help and contextual UI	Yes	Yes	UI renders; chat POST returns 410	No
Digital Human presentation	Yes	Simulated UI + adapter contract	UI prototype	No
Sessions	Yes	Untracked broker + schema	Not wired to UI	No distributed proof
Conversations	Yes	Schema	Not wired end to end	No
Support cases/evidence/resolution	Yes	Untracked platform + schema	Not wired to UI	No authoritative domain E2E
Tool gateway	Yes	Untracked gateway + demo registry	Not wired to UI	No production domain tools
Deterministic policy	Yes	Untracked engine	Called by demo tools	No production threshold approval
Knowledge retrieval	Metadata model	No full retrieval service found	No	No
Security/guardrails	Yes	Multiple layers	Legacy command layer only, endpoint unavailable	Targeted local evidence
PWA/Capacitor	Intended/deferred	Diagnostics only	No module-specific runtime proof	No
Deployment/rollback	Documented	Config and Azure direction exist	Integration incomplete	No Digital Human live proof

20. Gap Register and Recommended Roadmap

20.1 Release-Blocking Gaps

1. Restore a reachable authenticated customer-support API after the 410 migration.
2. Select one provider/orchestrator architecture and reconcile Google Gemini, Azure OpenAI, mock, and provider-mode documentation.
3. Commit, review, and test the untracked shared-core implementation or remove it from claimed release scope.
4. Replace mock booking/claim/dispute/KYC/insurance/context records with authoritative domain-service adapters.
5. Implement conversation persistence and cross-channel continuity.
6. Implement production-grade session tokens, distributed replay/idempotency, quotas, expiry, and circuit state.
7. Make policy enforcement non-bypassable and replace hard-coded monetary thresholds with approved versioned configuration.
8. Complete entity-link ownership, schema relationships, and transactional resolution verification.
9. Implement or formally defer the knowledge retrieval service.
10. Resolve governance contradictions and regenerate controlled manifest/evidence after fresh acceptance.

20.2 Digital Human Activation Gaps

1. Choose and contract a provider.
2. Implement session/media methods and token exchange.
3. Obtain explicit microphone/camera consent and browser permission.
4. Define biometric/voice/media data processing, retention, deletion, and regional controls.
5. Add real transcript and accessible captions.
6. Preserve conversation/case across provider and text fallback.
7. Add provider health, timeout, retry, circuit, cost, and revocation.
8. Complete browser, device, network-degradation, privacy, security, and accessibility acceptance.

20.3 Quality and Operations Gaps

- Convert demonstration scripts into maintained Jest/Playwright suites with assertions.
- Add observability across frontend, Azure API, provider, tool, policy, case, and domain services.
- Implement follow-up/SLA workers.
- Add explicit case category/evidence requirements.
- Add pagination and protected export to monitoring.
- Define SLOs, incident ownership, safe-hold escalation destinations, and user communication.

- Add retention and data-subject workflows for messages, logs, evidence, and provider media.
- Remove duplicate/overlapping guardrail, telemetry, and circuit-breaker implementations.

20.4 Recommended Delivery Sequence

Stage	Outcome
1. Baseline reconciliation	One tracked module tree, one provider decision, corrected ledgers, approved change request.
2. Text path restoration	Authenticated Azure route, safe orchestrator, persisted conversation, working Help/context UI.
3. Read-only grounding	Versioned knowledge and authoritative read-only tools with citations and privacy controls.
4. Case automation	Durable cases, evidence, follow-ups, safe holds, and user-visible status.
5. Consequential tools	Confirmation grants, step-up, versioned policy, transactional idempotency, post-verification.
6. Digital Human sandbox	Real provider behind identical shared core with full fallback.
7. Acceptance and rollout	Clean build/migration, security/privacy/accessibility/load tests, canary, evidence, new freeze.

21. API and Interface Contracts

21.1 Recommended Chat Request

```
{
  "version": "1",
  "sessionId": "ais_...",
  "conversationId": "aic_...",
  "channel": "help",
  "botId": "RENTipid Concierge Bot",
  "message": "I need help with my booking",
  "clientContext": {
    "route": "/dashboard/renter/bookings/bk_...",
    "bookingId": "bk_..."
  },
  "requestId": "uuid"
}
```

User ID and role must not be accepted as authority in this body. They are resolved from the authenticated server context.

21.2 Recommended Chat Response

```
{
  "version": "1",
  "requestId": "uuid",
  "sessionId": "ais_...",
  "conversationId": "aic_...",
  "caseId": "cas_...",
  "status": "answered",
  "message": "...",
  "cards": [],
  "pendingAction": null,
  "fallback": { "active": false, "reason": null }
}
```

For consequential actions, return a server-generated preview and confirmation grant rather than accepting a plain Boolean confirmation.

21.3 Error Contract

Use stable codes without internal leakage: `AUTH_REQUIRED`, `ACCOUNT_BLOCKED`, `SESSION_EXPIRED`, `RATE_LIMITED`, `CONTEXT_DENIED`, `TOOL_DENIED`, `CONFIRMATION_REQUIRED`, `STEP_UP_REQUIRED`, `POLICY_HOLD`, `PROVIDER_UNAVAILABLE`, `DOMAIN_UNAVAILABLE`, `VALIDATION_FAILED`, and `INTERNAL_ERROR`. Include a correlation/request ID and safe next action.

22. Glossary

Term	Definition
Autonomous support	Automated understanding and resolution within explicitly permitted, deterministic, auditable boundaries.
Digital Human	Avatar/voice/media presentation channel over the same shared support core.
Safe hold	A persisted stop state caused by uncertainty, conflict, threshold, missing evidence, or external authority.
Step-up	Stronger authentication/authorization required for a higher-risk action.
Tool	A server-side, typed adapter that gives the AI narrowly scoped access to an authoritative capability.
Request fingerprint	Stable identity used to detect replay and duplicate mutation.
Post-action verification	Read-back of authoritative state after an action before reporting success.
Contextual AI	Assistant launched with minimal server-authorized route/entity context.
Degraded mode	Text service remains available while media/Digital Human is disabled or unavailable.
Frozen baseline	Accepted commit/tag/artifact set changeable only through controlled change request.

23. Source Register

23.1 Controlled Module Records

`BASELINE.md`, `ARCHITECTURE_LOCK.md`, `IMPLEMENTATION_REGISTRY.md`, `INTERFACE_CONTRACTS.md`, `REQUIREMENTS_TRACEABILITY.md`, `AI_SERVICE_ACTION_MATRIX.md`, `AI_CASE_STATE_MODEL.md`, `TOOL_REGISTRY.md`, `POLICY_CATALOG.md`, `PROVIDER_REGISTER.md`, `SECURITY_PRIVACY_REVIEW.md`, `PRODUCTION_ACTIVATION.md`, `ROLLBACK_VERIFICATION.md`, `EVIDENCE_LEDGER.md`, `FINAL_ACCEPTANCE.md`, `CLOSURE_CERTIFICATE.md`, `FROZEN_SCOPE.md`, `CONTROLLED_ARTIFACT_MANIFEST.csv`, and `MASTER_CONTROL.md`.

23.2 Implementation and Data Sources

`src/lib/ai`, `src/components/ai`, `src/app/help/page.tsx`, `src/app/api/ai/chat/route.ts`, `src/app/dashboard/admin/ai-settings`, `src/app/dashboard/admin/ai-logs`, `apps/api/src/services/aiService.ts`, `prisma/schema.prisma`, and `migration 20260812120000_add_unified_ai_foundation`.

23.3 Validation Sources

`p3_test.ts`, `p4_test.ts`, `p5_test.ts`, `p6_test.ts`, `p7_test.ts`, `p8_test.ts`, `p9_test.ts`, `p10_test.ts`, `p11_test.ts`, and `run-p12-suite.ps1`.

23.4 Related Governance and Diagrams

`docs/governance/RENTipid-Master-Plan.md` and the rendered architecture, tool-gateway, and case-lifecycle diagrams in `docs/final-documentation/09-DIAGRAMS`.

24. Final Statement

RENTipid has a coherent vision for unified, autonomous, policy-bound customer service with a Digital Human presentation layer. The repository already contains much of the supporting schema, UI, security logic, policy logic, case lifecycle, administration, evidence, and rollback design. The system's next milestone is not more conceptual breadth; it is integration discipline: one tracked core, one reachable authenticated API, authoritative domain adapters, durable distributed controls, a real provider only when approved, and fresh end-to-end evidence.

Until those conditions are met, the accurate operating status is: historically accepted v1 foundation and degraded-mode design, with a currently unavailable frontend chat route, mock/simulated inference and media behavior, pending live Digital Human provider, and substantial shared-core work present outside current Git tracking.